



FACULTAD DE INGENIERÍA

REDES

(TA048) CURSO ALVAREZ - HAMELIN

Trabajo Práctico: File Transfer

[illegible]

Jonathan Dominguez Dídimo Paez Mateo Godoy Serrano
110057 98910 110912

Michelle Chen	Daiana Aldrey
105506	110624

Índice

1. Introducción	4
2. Hipótesis y suposiciones realizadas	5
3. Implementación	6
3.1. Arquitectura general	6
3.2. Estructura del Segmento	6
3.3. Hilos y Sockets	7
3.4. Cola de eventos	7
3.5. Máquina de estados	7
3.6. Sistema de temporización (Timer)	8
3.7. Conexión y Desconexión	8
3.7.1. Carga sin errores	9
3.8. Detalles del protocolo Selective Repite	10
3.8.1. Parámetros del Sistema	10
3.9. Stop and wait	11
3.10. Errores	11
3.10.1. Descargar archivo que esta siendo escrito	11
3.10.2. Descargar archivo inexistente	12
3.10.3. Cargar archivo existente que esta siendo procesado	13
3.10.4. Cargar archivo que supera el tamaño máximo	13
4. Pruebas	15
4.1. Wireshark	16
4.2. Casos de Error	18
5. Preguntas a responder	20
5.1. Describa la arquitectura Cliente-Servidor	20
5.2. ¿Cuál es la función de un protocolo de capa de aplicación?	20
5.3. Detalle el protocolo de aplicación desarrollado en este trabajo	20
5.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?	21
5.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.	21
5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?	21
6. Dificultades Encontradas	22
6.1. Definición de la Arquitectura de Concurrencia	22
6.2. Instalación de Mininet	22

7. Conclusión

23

1 Introducción

Este trabajo práctico está enfocado en crear una aplicación de red que tenga un protocolo RDT (Reliable Data Transfer) para la transferencia de archivos. Lo bautizamos con el nombre **RFTP**, por Reliable File Transfer Protocol, y ese es el nombre que tiene en el **plugin de Wireshark** implementado. Consta de una arquitectura cliente-servidor, la capa de transporte esta compuesta por un socket UDP y por encima está el protocolo mencionado.

Existen dos tipos de clientes: de carga y de descarga. El servidor provee archivos a los clientes de descarga y recibe archivos de los clientes de carga.

Los protocolos implementados son dos: de tipo Selective Repeat y de tipo Stop & Wait. Para simular problemas de red se utilizó Mininet, con una pérdida de paquetes del 10

El objetivo como bien mencionamos era asegurar las entregas, pero tambien implica manejar la concurrencia entre los distintos tipos de clientes que iríamos a tener. Como solución, se diseñaron manejadores de clientes en el servidor que funcionan como hilos; por cada conexión se lanzan tres hilos (detallados en las siguientes secciones) y se designa un puerto dedicado en el lado del servidor.

2 Hipótesis y suposiciones realizadas

Para la implementación, se asumieron las siguientes condiciones operativas para el correcto funcionamiento del sistema:

- **No se pueden subir archivos de mas de 20 MB:** El archivo mas grande que se puede cargar es 20 MB, si es mayor se le notifica al cliente, con una alerta de error, que esta superando el tamaño maximo.
- **No puede haber un cliente que suba un archivo y otro que descargue el mismo que se está subiendo ni viceversa:** No pueden operarse escrituras y lecturas sobre el mismo archivo a la vez. La aplicación da prioridad a quien primero empezó a escribir/leer y rechaza el resto de conexiones.
- **No puede haber más de un cliente que intente subir un archivo con igual nombre**
No pueden cargarse dos archivos con igual nombre, para que no haya problemas.
- **No contemplamos caídas inesperadas de conexión.**
- **La conexión, desconexión y el error se hace por StopAndWait:** Tanto la conexión, desconexión y error son paquetes por lo tanto, decidimos usar Stop and Wait para este caso en puntual. Permitiendo que el flujo de los paquetes dependa del protocolo del cliente, independientemente de esto mencionado.
- **UDP no fragmenta los segmentos del protocolo.** Asumimos un MTU de al menos 1400 bytes en el host de origen y el host de destino, de modo que un segmento del protocolo entre en un datagrama UDP sin fragmentarse.

3 Implementación

En esta sección se describe las decisiones de diseño, la arquitectura y los mecanismos principales utilizados para implementar el protocolo de transferencia sobre UDP. La solución se basa en una combinación de hilos, sockets, una cola de eventos, un timer y un diseño de máquina de estados para modelar el protocolo.

3.1 Arquitectura general

El servidor cuenta con tres tipos de protocolos: un protocolo **aceptador**, un protocolo **desconectador**, y un protocolo común. El cliente cuenta con dos: un protocolo **conectador** y uno común.

El protocolo aceptador es el encargado de aceptar conexiones nuevas, produciendo protocolos comunes que parten de un estado de sincronización inicial. Debido a que cada protocolo común se maneja en un socket dedicado, se necesitó de un protocolo desconectador que esté a la escucha de cierres de conexión para limpiar los protocolos comunes terminados.

El protocolo conectador es el que establece la conexión con el aceptador, produciendo un protocolo común que parte de un estado de sincronización inicial distinto al que produce el aceptador. El protocolo común es, justamente, común para cliente y servidor. En lo que reste de este informe, nos referiremos al protocolo común simplemente como *protocolo*, y a los demás protocolos los llamaremos por su nombre definido previamente.

Cada protocolo tiene una propia cola de eventos, a la que pueden llegar distintos tipos de eventos y de distintas fuentes.

El flujo general es el siguiente:

- Los datos llegan desde la red a través del socket.
- Un hilo receptor captura estos datos y los encapsula como eventos.
- Los eventos se insertan en la cola del protocolo.
- El estado del protocolo consume los eventos que llegan y toma un flujo.

3.2 Estructura del Segmento

La unidad fundamental de comunicación es el *segmento*. Este nombre fue decidido por ser el protocolo de transporte subyacente UDP, que pocas garantías da de la entrega de los paquetes, haciendo que el protocolo implementado parezca más de transporte que de aplicación. Sin embargo, el protocolo es de **aplicación**.

El segmento consiste de un encabezado fijo de 8 bytes y un *payload* variable. Su estructura es la siguiente:

Checksum (16 bits): Validación de integridad mediante suma de comprobación, descartando segmentos corruptos. Cada segmento expone métodos para calcular su propio checksum dado su contenido y para obtener el checksum almacenado en sus cabeceras, permitiendo verificarlos.

Número de Secuencia (16 bits): Identificador crítico para la reordenación en el receptor y la gestión de la ventana deslizante.

Longitud de Payload (11 bits): Define el tamaño de los datos útiles. Este campo tiene 11 bits por dos motivos: el tamaño máximo del payload es de 1392 bytes, que puede representarse con solo 11 bits; y la primera versión del protocolo solo tenía 5 bits de flags, por lo que se tenía un protocolo bien alineado a 2 bytes, pero al introducir el flag de error tuvo que rellenarse con bits extra para no tener que modificar la lógica ya implementada.

Flags (6 bits): (SYN, ACK, ERR, CTYP, PTYP FIN).

El checksum se calcula sumando cada par de bytes, tanto de las cabeceras como del payload, en big endian en un entero de 16 bits sin signo. Si la cantidad de bytes fuera impar, porque el payload tiene cantidad impar de bytes, el byte restante se suma directamente. Luego, se toma el complemento a 1 de ese número, y el resultado es el checksum almacenado en el campo de las cabeceras indicado.

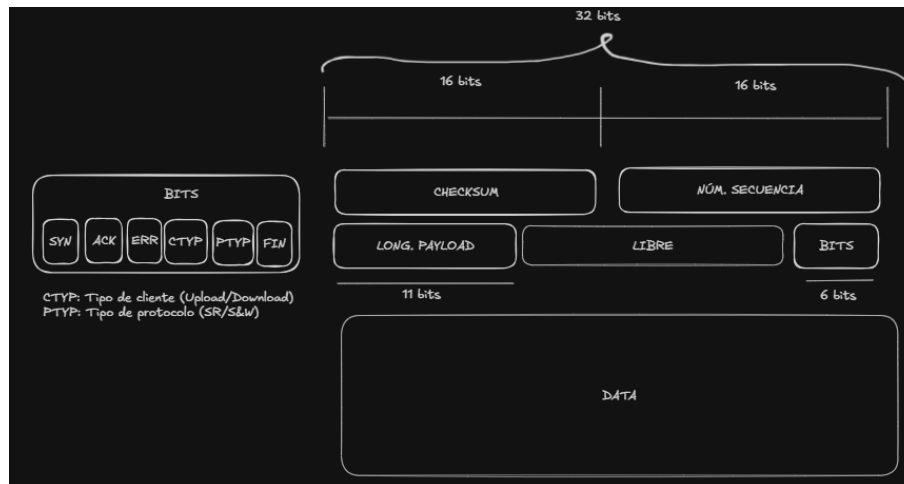


Figura 1: Segmento.

3.3 Hilos y Sockets

Los sockets utilizados para desarrollar el protocolo son sockets **UDP**, que solo garantizan multiplexación/demultiplexación y un mínimo control de integridad de datos con un checksum. La fiabilidad del protocolo fue implementada sobre este tipo socket.

Para permitir recepción y envío de datos de forma simultánea, se introducen un *receptor* y un emisor. En la práctica, el receptor se implementa como un **hilo**, y el emisor simplemente corre sobre el hilo de ejecución principal del protocolo.

Adicionalmente, se introduce un *temporizador*, que se implementa como un hilo que permite agendar notificaciones de eventos dado un tiempo (en segundos), una cola de eventos a la que notificar, y el evento en sí. Cada protocolo tiene su propio temporizador, por simplicidad.

En síntesis, por cada protocolo se tienen 3 hilos: un hilo principal de ejecución, un hilo de recepción, y un hilo temporizador.

3.4 Cola de eventos

Para comunicar hilos se utiliza una cola de mensajes, que es thread-safe por su implementación en Python. El hilo principal toma eventos de la cola, y el hilo temporizador y el receptor envían eventos a la misma; de este modo, el procesamiento de eventos se ejecuta en un único hilo, y no hay necesidad de introducir locks entre hilos de un mismo protocolo.

3.5 Máquina de estados

Un protocolo, por nuestra implementación, es una **máquina de estados**. Cada estado tiene su propio **contexto**, es decir, estado interno, que, en general, es el mismo para todos e incluye:

- El emisor
- El temporizador
- La cola de eventos
- El protocolo original

- Un *logger*, para debugging.

El protocolo funciona como una *máquina de Turing*, considerando que la *cinta* son los eventos recibidos por su cola de eventos. A grandes rasgos, un estado de protocolo ejecuta el siguiente flujo:

1. Espera un evento proveniente de la cola de eventos.
2. Ejecuta la lógica correspondiente según el evento recibido, que puede o no resultar en un cambio de estado.

Los estados principales son:

- **Sincronización (SyncState)**: Establecimiento de los datos de la conexión.
- **Flujo (FlowState)**: Transferencia de datos.
- **Finalización (FinState)**: Cierre controlado de la conexión.
- **Detención (HaltState)**: Estado *muerto*, el protocolo terminó su ejecución.

A su vez, el protocolo utilizado por el servidor tiene su propio estado de sincronización, y el utilizado por el cliente también. Descontando esta salvedad, ambos protocolos comparten el resto de estados.

Esto permite que, luego de transicionar del estado de sincronización al estado de flujo, el comportamiento del protocolo sea **igual para cliente y servidor**, por lo que a lo largo de este informe consideraremos que los protocolos de *carga* y *descarga* son opuestos (es decir, si el cliente tiene un protocolo de descarga, el servidor tiene uno de carga, y viceversa).

El estado de flujo es uno solo, implementado como *Selective Repeat* por defecto, pero se comporta como un protocolo *Stop Wait* fijando las ventanas de recepción y envío con tamaño en 1.

3.6 Sistema de temporización (Timer)

El sistema de temporizadores sigue un esquema de eventos programados. Cuando se envía un segmento, se registra un temporizador asociado. Si no se recibe un acuse de recibo antes del tiempo límite, se genera un evento de *timeout* que se envía a la cola de eventos. Si se recibe el acuse de recibo, es posible cancelar el temporizador.

El temporizador utilizado en la implementación final, por simplicidad, es un temporizador con espera activa, o **BusyWaitTimer**.

Este diseño permite manejar múltiples retransmisiones de manera concurrente. Cada cliente tiene un temporizador propio, compartido entre los estados.

3.7 Conexión y Desconexión

Según el tipo de cliente (de carga o de descarga) la conexión se establece en 2 o 3 pasos, lo que causa que el servidor y el cliente tengan estados de sincronización diferentes. Para el caso de cliente de **descarga**:

- El cliente envía un segmento SYN con el nombre del archivo a descargar.
- El servidor responde con SYN-ACK que tiene como payload el tamaño del archivo.
- El cliente confirma con SYN-ACK.

Sería posible que el cliente rechaze la conexión del servidor si el nombre del archivo es muy grande, pero descartamos ese caso porque consideramos que no aportaba más valor que los dos errores ya manejados.

El servidor rechaza la conexión si el archivo no existe o está siendo cargado por otro cliente. Si esto sucede, ni siquiera se crea el protocolo del lado del servidor, y no se entra al estado de sincronización.

Para el caso de cliente de **carga**:

- El cliente envía un segmento SYN con el tamaño y el nombre del archivo a cargar.
- El servidor responde con ACK.

El servidor rechaza la conexión si el archivo es demasiado grande o si ya está siendo cargado o descargado por otro cliente. Si el archivo es demasiado grande, ni siquiera se crea el protocolo del lado del servidor; si el archivo ya está siendo cargado o descargado, puede que se haya creado el protocolo al dispararse ese error, pero el protocolo está diseñado para poder ser abortado en cualquier instancia de la vida del mismo.

Luego de la sincronización, ambos protocolos (del lado del cliente y del servidor, sean de carga o de descarga) transicionan al estado de flujo, donde la lógica es intercambiable para ambos.

La desconexión sucede durante el FinState, y tiene la misma lógica para ambos protocolos (de carga y de descarga), y la implementación solo difiere en el estado interno del propio FinState:

- Se espera a recibir un evento de la cola de eventos.
- Si la *dirección* (del socket UDP) está vacía, se interpreta como un evento causado por el propio usuario del protocolo.
- Si se recibe un evento con dirección vacía, eso se resuelve a **enviar un segmento FIN al peer desconectador**.
- Cuando se recibe un segmento FIN del peer desconectador, identificado por su dirección no vacía, el estado de finalización transiciona al HaltState.

3.7.1. Carga sin errores

El siguiente diagrama representa un flujo *perfecto* de ejecución de un cliente de carga sin errores.

Las líneas sólidas se interpretan como paquetes enviados a través de la red que son temporizados del lado emisor para retransmitirlos si el temporizador llega a su fin. Las líneas semicontinuas que no son color magenta son segmentos no temporizados pero que pueden ser retransmitidos dados otros eventos.

Las líneas de color verde representan paquetes SYN. Las de color amarillo representan paquetes ACK. Las azules representan paquetes de payload. Las marrones representan paquetes FIN. Las magenta representan eventos ocurridos durante la ejecución que no son transmitidos por la red.

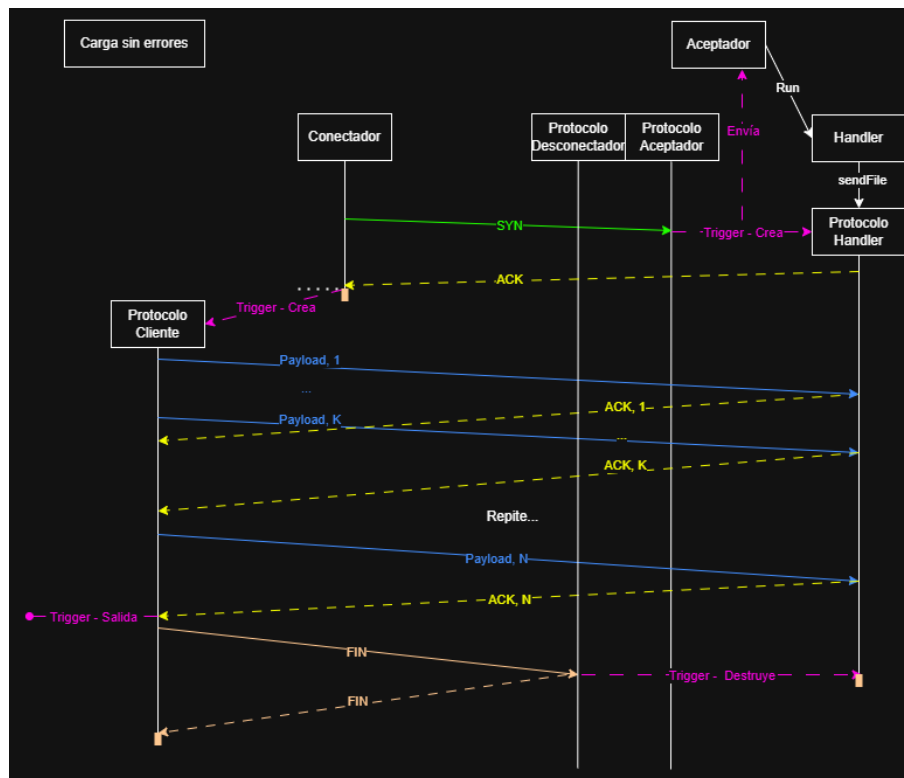


Figura 2: Carga sin errores

Descarga sin errores

El siguiente diagrama representa un flujo *perfecto* de ejecución de un cliente de descarga sin errores.

Los significados de los colores de las líneas es el mismo que en el caso de carga.

3.8 Detalles del protocolo Selective Repite

3.8.1. Parámetros del Sistema

- **WINDOW_SIZE (20):** Define la cantidad máxima de segmentos que el emisor puede tener sin confirmar simultáneamente.
- **WINDOW_RECEIVE_SIZE (20):** Tamaño del buffer del receptor. Su mayor capacidad respecto a la de emisión permite mitigar asincronías y evitar el descarte de paquetes fuera de orden.
- **RTO_SECONDS (0.05s):** Valor constante del *Retransmission Timeout* que rige la expiración de los temporizadores.

Arquitectura del Emisor

El proceso de transmisión gestiona la inyección de paquetes y la recuperación de errores de la siguiente manera:

- **Gestión de Ventana:** El emisor satura la ventana enviando ráfagas de segmentos. Cada paquete enviado genera una instancia de *PacketInFlight* asociada a un temporizador independiente basado en su número de secuencia.

- **Procesamiento de ACKs:** Al recibir una confirmación (ACK), se cancela el temporizador específico y se elimina el paquete de la estructura de control. La base de la ventana (*baseNumber*) se desliza de forma secuencial únicamente si el ACK recibido corresponde al segmento más antiguo pendiente.
- **Retransmisión Selectiva:** Ante la expiración de un temporizador, el protocolo reenvía **únicamente** el segmento perdido, optimizando el uso de la red al evitar la retransmisión innecesaria de la ventana completa.

Arquitectura del Receptor

El receptor está diseñado para manejar el desorden de paquetes y optimizar la escritura en disco implementando:

- **Buffer de Recepción:** Los segmentos que llegan fuera de orden (pero dentro de la ventana de recepción) se almacenan en un diccionario en memoria RAM, enviando el ACK correspondiente inmediatamente.
- **Mitigación de Duplicados:** Si se recibe un paquete con número de secuencia menor a la base esperada o ya existente en el buffer, el sistema descarta el *payload* pero re-emite el ACK para evitar bloqueos en el emisor.
- **Escritura Contigua en Lote (*Batch*):** Para optimizar operaciones de I/O, el receptor verifica la contigüidad de los segmentos desde la base. Al completar una secuencia, extrae los datos y realiza una única escritura a disco por lote.
- **Cierre Diferido:** Tras recibir el flag FIN, el estado permanece activo hasta que los bytes escritos en disco coincidan exactamente con el tamaño nominal del archivo, garantizando la integridad de la descarga.

3.9 Stop and wait

En la teoría de protocolos de red, *Stop & Wait* (SW) y *Selective Repeat* (SR) no son entidades mutuamente excluyentes. La principal diferencia radica en el tamaño de la ventana de transmisión (W). Mientras que un protocolo SR tradicional utiliza una ventana $W > 1$ para aprovechar el *pipeline* del canal, el protocolo *Stop & Wait* se define estrictamente como un protocolo de ventana deslizante donde la ventana de emisión y recepción es de tamaño unitario ($W = 1$). Bajo esta premisa teórica, se determinó que la implementación más eficiente para el sistema es restringir el tamaño de la ventana que se usa en *Selective Repeat* a 1.

Al configurar el protocolo de Repetición Selectiva (SR) con un tamaño de ventana igual a 1, el sistema emula con exactitud el comportamiento de Stop & Wait (SW). En este estado, el emisor queda bloqueado tras enviar un único paquete, permitiendo una nueva transmisión solo tras recibir la confirmación (ACK) o al retransmitir el segmento tras un timeout.

3.10 Errores

A continuación se describe el flujo que maneja los errores posibles.

3.10.1. Descargar archivo que esta siendo escrito

Cuando un cliente intenta realizar la descarga de un archivo que se encuentra bloqueado por estar siendo escrito simultáneamente por otro proceso, el sistema prevé dos posibles mecanismos de resolución:

Validación Temprana La detección del bloqueo se realiza en la etapa más temprana posible. El conector del cliente envía el segmento SYN solicitando el archivo. El protocolo aceptador del servidor valida el estado del archivo antes de asignar recursos o despachar hilos. Al detectar que el

archivo está siendo escrito, rechaza inmediatamente la conexión respondiendo con un segmento de error explícito (ERR), informando al cliente de la indisponibilidad temporal del recurso sin llegar a establecer una conexión.

Este mecanismo no es una prevención directa, sino una consecuencia de la necesidad de verificar los datos del archivo antes de establecer la conexión. Puede suceder que al revisar estos datos el archivo aún no esté siendo escrito, pero sí al entrar al estado de flujo. Ese caso sí es manejado intencionalmente.

Fallo en Instanciación Si al verificar la disponibilidad del archivo previo al estado de sincronización el archivo estaba libre, pero al entrar al estado de flujo ya estaba tomado, eso causará una excepción en el administrador de archivos del servidor, causando que el estado de flujo pase directamente al estado de finalización directamente.

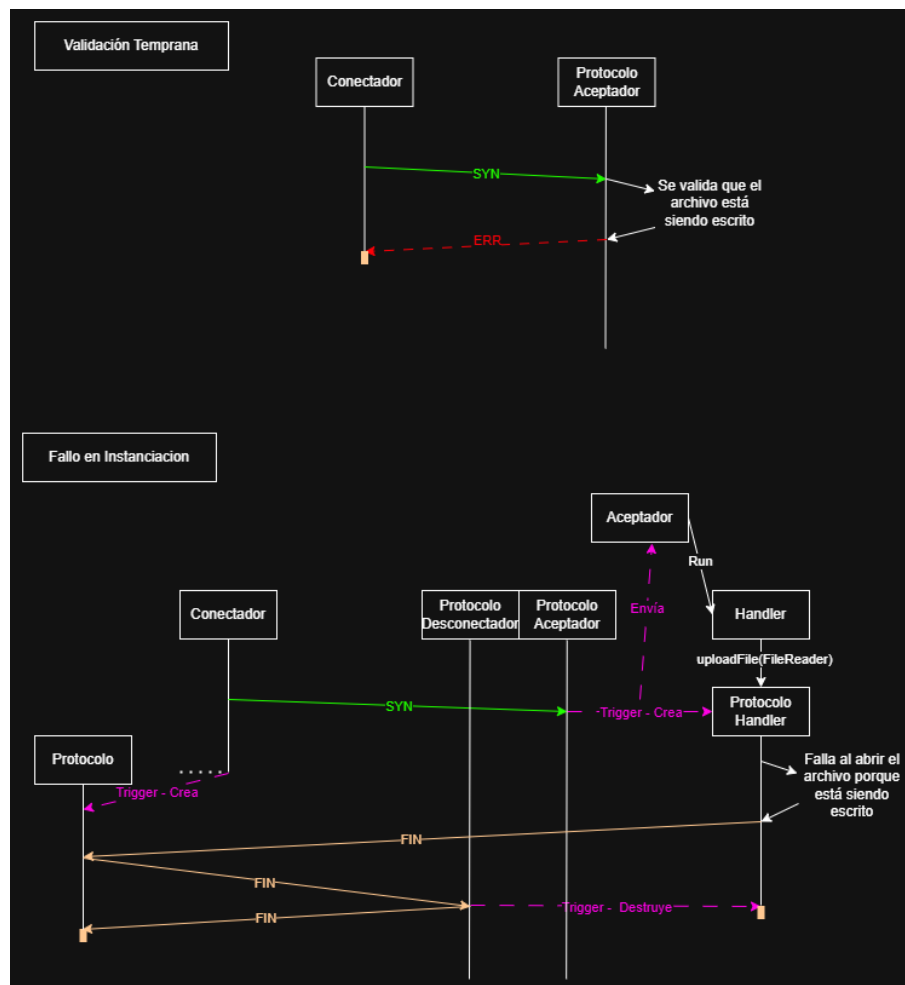


Figura 4: Error descargar archivo que esta siendo escrito

3.10.2. Descargar archivo inexistente

Si el archivo a descargar directamente no existe, el protocolo aceptador rechaza la conexión enviando un segmento de ERR al cliente con el mensaje.

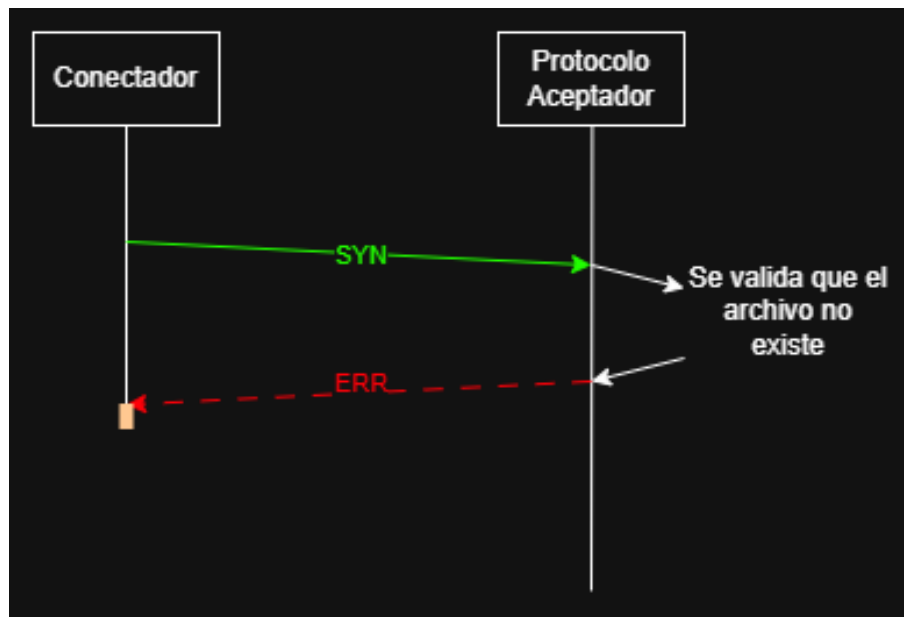


Figura 5: Error descargar archivo inexistente

3.10.3. Cargar archivo existente que esta siendo procesado

Similar al caso de descarga de archivo que está siendo cargado, puede suceder que dos clientes quieran cargar un mismo archivo a la vez, o que uno quiera cargarlo y el otro descargar una versión previa del archivo del servidor a la vez. En ambos casos, los mecanismos de prevención son los mismos que para el caso de descarga, solo que esta vez se prohíbe la carga tanto si el otro cliente está cargando como leyendo el archivo en disputa.

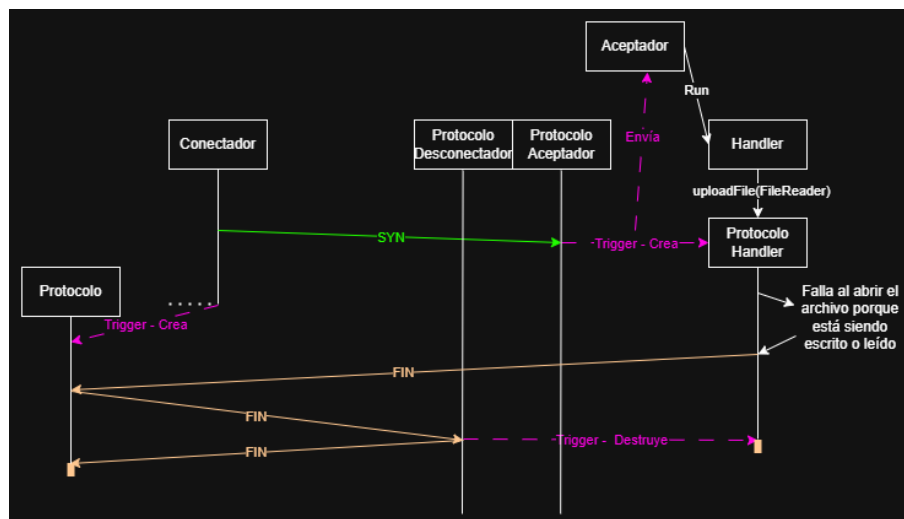


Figura 6: Error cargar archivo mientras se escribe o lee

3.10.4. Cargar archivo que supera el tamaño máximo

Cuando un cliente intenta cargar un archivo cuyo tamaño excede el límite máximo permitido (20 MiB), se rechaza la conexión desde el protocolo aceptador con un mensaje de error.

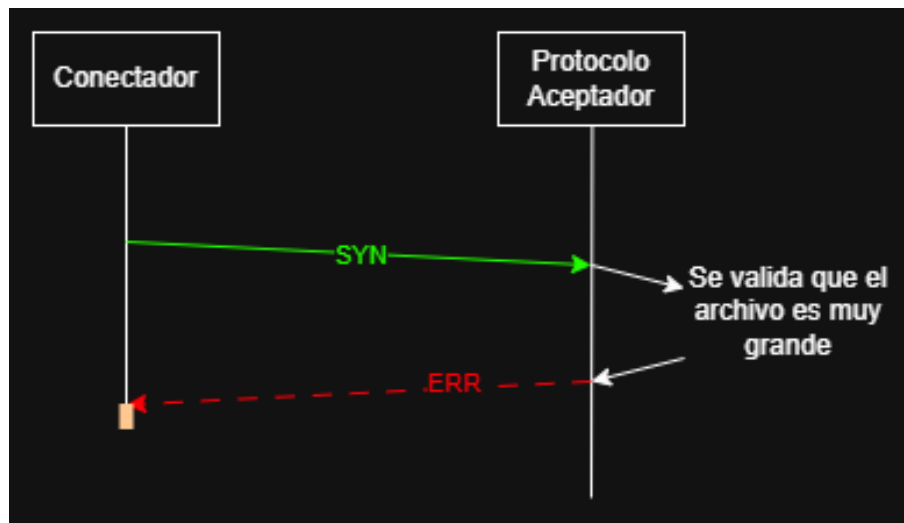


Figura 7: Error cargar archivo que supera tamaño máximo

4 Pruebas

Las pruebas realizadas se llevaron a cabo mediante Mininet y Wireshark.

Mininet

Se tienen archivos de 5MB con los siguientes nombres:

prueba-a.txt	5.0 MB	Today 19:44	☆
prueba-b.txt	5.0 MB	Today 19:44	☆
prueba-c.txt	5.0 MB	Today 19:44	☆
prueba-d.txt	5.0 MB	Today 19:44	☆

Figura 8: Tamaño de archivos

La carpeta `storage_server` corresponde al servidor y contiene los archivos que se van a descargar, en ella también se almacenaran los archivos subidos. Mientras que la carpeta `storage_client` es la que almacenara las descargas. Inicialmente empiezan así:

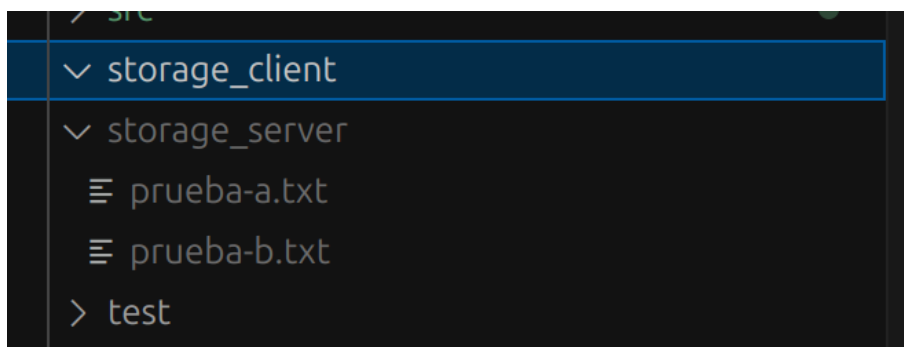


Figura 9: Carpetas iniciales

Se abren 5 nodos con mininet con pérdida de 10% de los cuales, el h1 se usa para levantar el servidor, el h2 para cargar con sr, el h3 para cargar con sw, el h4 para descargar con sr y el h5 para descargar con sw. Así se prueba la concurrencia y el uso de ambos protocolos para descarga y carga.

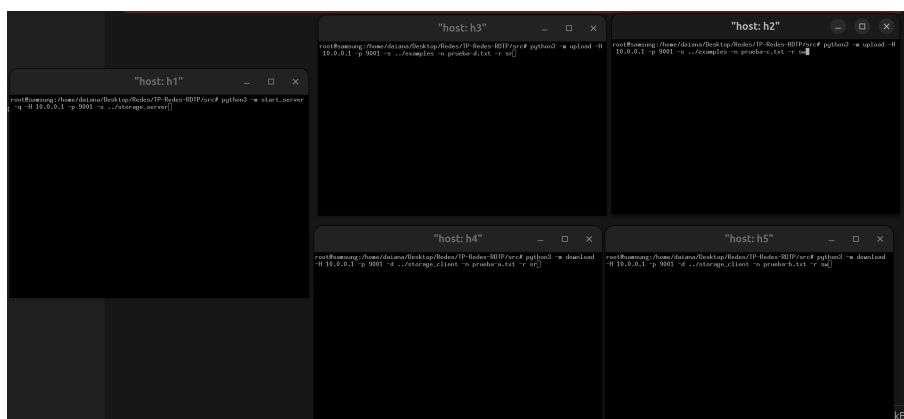


Figura 10: Terminales iniciales

Al terminar de ejecutarse todos los clientes se observa que los procesos fueron realizados con éxito.

```

"host: h3"
Re-sending segment... Seq: 3627
Received ACK for seq=3606
Received ACK for seq=3607
Received ACK for seq=3609
Received ACK for seq=3611
Received ACK for seq=3619
Received ACK for seq=3626
Received ACK for seq=3627
Timeout for seq: 3618
Re-sending segment... Seq: 3618
Timeout for seq: 3623
Re-sending segment... Seq: 3623
Timeout for seq: 3625
Re-sending segment... Seq: 3625
Received ACK for seq=3618
Received ACK for seq=3623
Timeout for seq: 3625
Re-sending segment... Seq: 3625
Received ACK for seq=3625
File sent successfully
Preparing to close file reader
Transitioning to FinState seq=3628
[CLIENT] ELAPSED: 30.13972005100388
root@samsung:/home/daiana/Desktop/Redes/IP-Redes-ROTP/src#

"host: h2"
Timeout for seq: 3619
Re-sending segment... Seq: 3619
Received ACK for seq=3619
Received ACK for seq=3620
Timeout for seq: 3621
Re-sending segment... Seq: 3621
Received ACK for seq=3621
Timeout for seq: 3622
Re-sending segment... Seq: 3622
Received ACK for seq=3622
Received ACK for seq=3623
Timeout for seq: 3624
Re-sending segment... Seq: 3624
Received ACK for seq=3624
Timeout for seq: 3625
Re-sending segment... Seq: 3625
Received ACK for seq=3625
Received ACK for seq=3626
Received ACK for seq=3627
File sent successfully
Preparing to close file reader
Transitioning to FinState seq=3628
[CLIENT] ELAPSED: 196.74003276993204
root@samsung:/home/daiana/Desktop/Redes/IP-Redes-ROTP/src#

"host: h4"
Received DATA seq=3624, len=1392
Sent ACK seq=3624
Ready to write seq=3624
Written 1392 bytes, Total downloaded=5044608
Received DATA seq=3625, len=1392
Sent ACK seq=3625
Ready to write seq=3625
Ready to write seq=3626
Ready to write seq=3627
Written 2900 bytes, Total downloaded=5047508
Transitioning to FinState seq=3627, Re-ACKing
Sent ACK seq=3627
Duplicate old packet seq=3624, Re-ACKing
Sent ACK seq=3624
Duplicate old packet seq=3626, Re-ACKing
Sent ACK seq=3626
Received FIN seq=3630
Transitioning to FinState seq=0
Closing file writer
CLOSING
FLUSING
CLOSING
[CLIENT] ELAPSED: 31.94517915799952
root@samsung:/home/daiana/Desktop/Redes/IP-Redes-ROTP/src#

"host: h5"
Received DATA seq=3624, len=1392
Sent ACK seq=3624
Ready to write seq=3624
Written 1392 bytes, Total downloaded=5044608
Received DATA seq=3625, len=1392
Sent ACK seq=3625
Ready to write seq=3625
Written 1392 bytes, Total downloaded=5046000
Received DATA seq=3626, len=1392
Sent ACK seq=3626
Ready to write seq=3626
Written 1392 bytes, Total downloaded=5047392
Received DATA seq=3627, len=116
Sent ACK seq=3627
Ready to write seq=3627
Written 116 bytes, Total downloaded=5047508
Received FIN seq=3630
Transitioning to FinState seq=0
Closing file writer
CLOSING
FLUSING
CLOSING
[CLIENT] ELAPSED: 193.09860479000054
root@samsung:/home/daiana/Desktop/Redes/IP-Redes-ROTP/src#

```

Figura 11: Terminales finales

Se comprueba que los archivos fueron creados y alojados en las carpetas correctas.

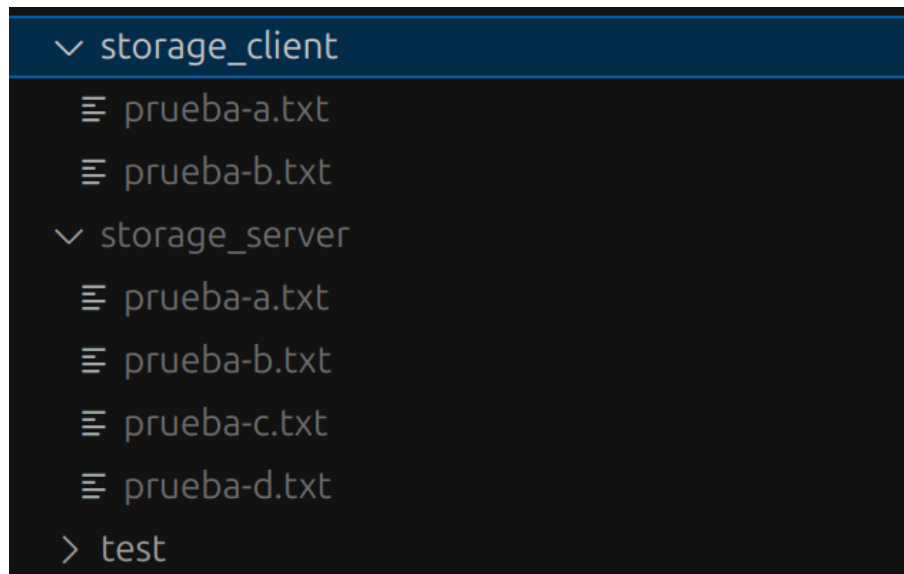


Figura 12: Carpetas Finales

4.1 Wireshark

Se capturó en Wireshark luego de correr las 5 terminales (la del servidor y los 4 clientes) desde el servidor h1 para capturar la información de los clientes. Se tomaron a continuación capturas de todos los clientes ejecutados y se observa al final la llegada del fin, ya que abajo de todo el paquete es de 8 bytes mientras que los anteriores eran mucho más grandes por transmitir información de payload.

Se filtró con 10.0.0.2 ya que h2 corre al cliente que ejecuta upload con Stop and Wait y se observa que la ventana es de tamaño 1, ya que el tamaño enviado por el cliente siempre es de length 50:

No.	Time	Source	Destination	Protocol	Length	Source Port	Destination Port	Info
35091	195.319255618	10.0.0.2	10.0.0.1	UDP	1442	44368	34189	44368 → 34189 Len=1400
35092	195.328635793	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35093	195.321838110	10.0.0.2	10.0.0.1	UDP	1442	44368	34189	44368 → 34189 Len=1400
35094	195.323249968	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35095	195.324396504	10.0.0.2	10.0.0.1	UDP	1442	44368	34189	44368 → 34189 Len=1400
35096	195.325858159	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35112	195.516813636	10.0.0.2	10.0.0.1	UDP	1442	44368	34189	44368 → 34189 Len=1400
35113	195.518199048	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35114	195.519881178	10.0.0.2	10.0.0.1	UDP	1442	44368	34189	44368 → 34189 Len=1400
35115	195.520132728	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35116	195.520479252	10.0.0.2	10.0.0.1	UDP	166	44368	34189	44368 → 34189 Len=124
35117	195.520798757	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35122	195.817673918	10.0.0.2	10.0.0.1	UDP	166	44368	34189	44368 → 34189 Len=124
35123	195.818297733	10.0.0.1	10.0.0.2	UDP	50	34189	44368	34189 → 44368 Len=8
35124	195.819835747	10.0.0.2	10.0.0.1	UDP	50	44368	9002	44368 → 9002 Len=8
35125	195.820117831	10.0.0.1	10.0.0.2	UDP	50	9002	44368	9002 → 44368 Len=8

Figura 13: Upload sw

Se filtró con 10.0.0.3 para ver los paquetes de h3 que es el cliente que ejecuta upload con Selective Repite y se observa que el tamaño de los archivo llega a ser de length 1442 y length 1447, lo que muestra que la ventana cambia:

No.	Time	Source	Destination	Protocol	Length	Source Port	Destination Port	Info
17580	26.083256504	10.0.0.3	10.0.0.1	UDP	1442	40097	33898	40097 → 33898 Len=1400
17581	26.083569753	10.0.0.3	10.0.0.1	UDP	1442	40097	33898	40097 → 33898 Len=1400
17582	26.083669314	10.0.0.3	10.0.0.1	UDP	1442	40097	33898	40097 → 33898 Len=1400
17583	26.08321926	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17584	26.088479292	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17585	26.088615700	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17586	26.088713678	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17645	26.183219541	10.0.0.3	10.0.0.1	UDP	1442	40097	33898	40097 → 33898 Len=1400
17647	26.183379244	10.0.0.3	10.0.0.1	UDP	1442	40097	33898	40097 → 33898 Len=1400
17648	26.184583571	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17649	26.185784489	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17687	26.283356373	10.0.0.3	10.0.0.1	UDP	1442	40097	33898	40097 → 33898 Len=1400
17688	26.285829226	10.0.0.1	10.0.0.3	UDP	50	33898	40097	33898 → 40097 Len=8
17689	26.287075411	10.0.0.3	10.0.0.1	UDP	50	40097	9002	40097 → 9002 Len=8
17690	26.287516345	10.0.0.1	10.0.0.3	UDP	50	9002	40097	9002 → 40097 Len=8
17736	26.383385769	10.0.0.3	10.0.0.1	UDP	50	40097	9002	40097 → 9002 Len=8
17737	26.383841214	10.0.0.1	10.0.0.3	UDP	50	9002	40097	9002 → 40097 Len=8

Figura 14: Upload sr

Se filtró con 10.0.0.4 para ver los paquetes de h4 que es el cliente que ejecuta download con Selective Repite:

No.	Time	Source	Destination	Protocol	Length	Source Port	Destination Port	Info
20587	32.854612859	10.0.0.1	10.0.0.4	UDP	1442	41749	35980	41749 → 35980 Len=1400
20588	32.854756523	10.0.0.1	10.0.0.4	UDP	1442	41749	35980	41749 → 35980 Len=1400
20589	32.854886698	10.0.0.1	10.0.0.4	UDP	1442	41749	35980	41749 → 35980 Len=1400
20595	32.861197778	10.0.0.1	10.0.0.4	UDP	50	35980	41749	35980 → 41749 Len=8
20596	32.861374633	10.0.0.4	10.0.0.1	UDP	50	35980	41749	35980 → 41749 Len=8
20597	32.861599639	10.0.0.4	10.0.0.1	UDP	50	35980	41749	35980 → 41749 Len=8
20598	32.862155680	10.0.0.4	10.0.0.1	UDP	50	35980	41749	35980 → 41749 Len=8
20603	32.955512738	10.0.0.1	10.0.0.4	UDP	1442	41749	35980	41749 → 35980 Len=1400
20604	32.955889834	10.0.0.1	10.0.0.4	UDP	1442	41749	35980	41749 → 35980 Len=1400
20605	32.956012456	10.0.0.1	10.0.0.4	UDP	1442	41749	35980	41749 → 35980 Len=1400
20613	32.961192389	10.0.0.4	10.0.0.1	UDP	50	35980	41749	35980 → 41749 Len=8
20614	32.961337865	10.0.0.4	10.0.0.1	UDP	50	35980	41749	35980 → 41749 Len=8
20615	32.961477167	10.0.0.4	10.0.0.1	UDP	50	35980	41749	35980 → 41749 Len=8
20618	32.964548349	10.0.0.1	10.0.0.4	UDP	50	41749	35980	41749 → 35980 Len=8
20619	32.965868555	10.0.0.4	10.0.0.1	UDP	50	35980	9002	35980 → 9002 Len=8
20620	32.966393979	10.0.0.1	10.0.0.4	UDP	50	9002	35980	9002 → 35980 Len=8

Figura 15: Download de sr

Se filtró con 10.0.0.5 para ver los paquetes de h5 que es el cliente que ejecuta download con Stop and Wait:

No.	Time	Source	Destination	Protocol	Length	Source Port	Destination Port	Info
35400	202.574658260	10.0.0.1	10.0.0.5	UDP	50	53174	51815	53174 -> 51815 Len=8
35401	202.575178166	10.0.0.1	10.0.0.5	UDP	1442	51815	53174	51815 -> 53174 Len=1400
35402	202.671318012	10.0.0.1	10.0.0.5	UDP	1442	51815	53174	51815 -> 53174 Len=1400
35403	202.771494563	10.0.0.1	10.0.0.5	UDP	1442	51815	53174	51815 -> 53174 Len=1400
35404	202.772793342	10.0.0.1	10.0.0.5	UDP	50	53174	51815	53174 -> 51815 Len=8
35405	202.773643560	10.0.0.1	10.0.0.5	UDP	1442	51815	53174	51815 -> 53174 Len=1400
35406	202.774718830	10.0.0.1	10.0.0.5	UDP	50	53174	51815	53174 -> 51815 Len=8
35407	202.871941680	10.0.0.1	10.0.0.5	UDP	1442	51815	53174	51815 -> 53174 Len=1400
35408	202.874456958	10.0.0.1	10.0.0.5	UDP	50	53174	51815	53174 -> 51815 Len=8
35409	202.875269564	10.0.0.1	10.0.0.5	UDP	1442	51815	53174	51815 -> 53174 Len=1400
35410	202.876422241	10.0.0.1	10.0.0.5	UDP	50	53174	51815	53174 -> 51815 Len=8
35411	202.972164042	10.0.0.1	10.0.0.5	UDP	166	51815	53174	51815 -> 53174 Len=124
35412	202.973259093	10.0.0.1	10.0.0.5	UDP	50	53174	51815	53174 -> 51815 Len=8
35413	202.974456530	10.0.0.1	10.0.0.5	UDP	50	51815	53174	51815 -> 53174 Len=8
35414	202.976056088	10.0.0.1	10.0.0.5	UDP	50	53174	9002	53174 -> 9002 Len=8
35415	202.976317489	10.0.0.1	10.0.0.5	UDP	50	9002	53174	9002 -> 53174 Len=8

Figura 16: Download de sw

4.2 Casos de Error

Subir un archivo mayor a 20 MB

Si se intenta subir un archivo de tamaño mayor a 20 MB se envía mensaje de error al cliente:

```
Arguments: (Clib.event.segment_timeout_event.SegmentTimeoutEvent object at 0x72cc/b9e0860),)
Server error: File prueba-z.txt is too large
[CLIENT] ELAPSED: 1.1113121779999346
root@samsung:/home/daiana/Desktop/Redes/TP-Redes-RDTP/src#
```

Figura 17: Error archivo muy grande

Descargar un archivo inexistente

Si se intenta descargar un archivo que el servidor no posee, se envía mensaje de error alertando que el archivo no existe:

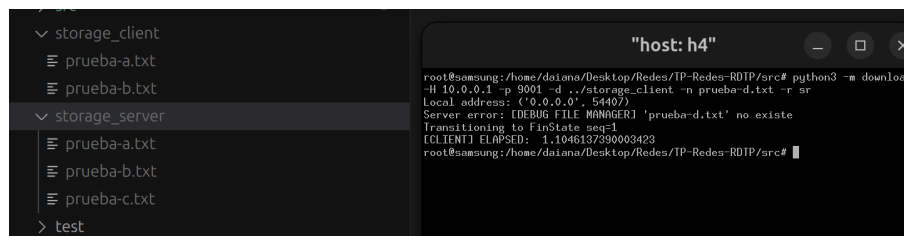


Figura 18: Error archivo inexistente

Descargar un archivo que se esta subiendo

Si se intenta descargar un archivo que se esta subiendo, se envía mensaje de error avisandole al cliente que dicho archivo no se subio todavía. En la siguiente imagen se observa que el cliente h3 esta subiendo el archivo prueba-d.txt, por lo que el cliente h4 recibe error al querer descargar dicho archivo cuando no termino de completarse la subida.

```
"host: h3"
Re-sending segment... Seq: 2217
Received ACK for seq=2228
Received ACK for seq=2230
Received ACK for seq=2217
Received ACK for seq=2234
Timeout for seq: 2218
Re-sending segment... Seq: 2218
Timeout for seq: 2193
Re-sending segment... Seq: 2193
Received ACK for seq=2218
Received ACK for seq=2236
Received ACK for seq=2237
Received ACK for seq=2238
Timeout for seq: 2213
Re-sending segment... Seq: 2213
Timeout for seq: 2215
Re-sending segment... Seq: 2215
Received ACK for seq=2239
Received ACK for seq=2213
Received ACK for seq=2215
Received ACK for seq=2240
Received ACK for seq=2241
Received ACK for seq=2242
Received ACK for seq=2243
Received ACK for seq=2246
Timeout for seq: 2227
Re-sending segment... Seq: 2227

"host: h4"
root@samsung:/home/daiana/Desktop/Redes/TP-Redes-RDTP/src# python3 -m download -H 10.0.0.1 -p 9
001 -d ../storage_client -n prueba-d.txt -r sr
Local address: ('0.0.0.0', 50056)
Server error: [DEBUG FILE MANAGER] 'prueba-d.txt aun no está escrito por completo'
Transitioning to FinState seq=1
[CLIENT] ELAPSED: 1.0041073669999605
root@samsung:/home/daiana/Desktop/Redes/TP-Redes-RDTP/src#
```

Figura 19: Error archivo no subido todavía

5 Preguntas a responder

5.1 Describa la arquitectura Cliente-Servidor

En una arquitectura cliente-servidor, hay dos partes:

- Servidor: quien provee un servicio, de ahí el nombre. Tiene un recurso o funcionalidad al que un externo quiere acceder, y provee mecanismos para ello. Generalmente *escucha* conexiones entrantes.
- Cliente: quien busca acceder al recurso o funcionalidad provistos por el servidor. Generalmente *se conecta* al servidor.

En este trabajo, hay dos tipos de clientes: de carga y de descarga. El servidor maneja ambos tipos de clientes.

5.2 ¿Cuál es la función de un protocolo de capa de aplicación?

Un Protocolo de la capa de aplicación permite intercambiar datos entre dos aplicaciones de red. Define 4 elementos:

- Los tipos de mensajes que se intercambian.
- La Sintaxis de los mensajes definidos, los campos que tienen y cómo se separan.
- La semántica de esos campos, es decir, qué significa la información que llevan.
- Las reglas para determinar cuándo y cómo un proceso envía mensajes y responde a ellos

5.3 Detalle el protocolo de aplicación desarrollado en este trabajo

El protocolo de aplicación desarrollado se construye sobre UDP y garantiza la transferencia confiable de archivos, implementando un mecanismo de máquina de estados. El protocolo permite cargar y descargar archivos con el servidor y determinar el tipo de protocolo a utilizar (Selective Repeat o Stop Wait) según su configuración inicial. Se implementa con hilos.

La arquitectura fue desarrollada en profundidad a lo largo del informe. Lo más relevante es que el protocolo expone cinco funcionalidades:

- Aceptar conexiones (accept).
- Terminar conexiones aceptadas (disconnect).
- Establecer conexiones (connect).
- Enviar archivos (send).
- Recibir archivos (download)

Por debajo, la estrategia de envío y recepción de datos utiliza una ventana deslizante que puede o no ser de tamaño 1 para forzar un Stop Wait, y admite retransmisiones por timeouts.

Las conexiones se establecen a través de un proceso de sincronización, que difiere entre cliente y servidor y si el cliente es de carga o de descarga. Cada tipo de sincronización (cliente o servidor) está enmarcado en un estado propio, pero ambos transicionan hacia el mismo estado de flujo, que a su vez transiciona hacia un único estado de finalización, o de detención si el cierre debe ser inminente.

El estado de finalización maneja la desconexión limpia, y transiciona hacia el estado de detención. El estado de detención simplemente representa un protocolo cuyo ciclo de vida ha terminado, y su nombre (HaltState) fue tomado de uno de los conceptos de las máquinas de Turing.

5.4 La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

TCP y UDP extienden el servicio host a host de IP a un servicio proceso a proceso, mediante **multiplexación y demultiplexación**. Ambos incluyen comprobación de errores (checksum) en sus segmentos.

UDP ofrece dos servicios básicos: Entrega de datos proceso a proceso y comprobación de errores. No ofrece transferencia fiable, es decir, no garantiza la entrega ni que el orden de llegada coincida con el orden de envío. Tampoco provee mecanismos de control de flujo o de congestión.

TCP ofrece, además de lo mismo que UDP, transferencia fiable de datos: garantiza que los datos enviados por el proceso emisor sean entregados al proceso receptor correctamente y en orden. Además, TCP proporciona un mecanismo de control de flujo y uno de control de congestión.

TCP es un protocolo orientado a conexión, mientras que UDP no. Una conexión (TCP o no) es identificada por IP de origen, IP de destino, puerto de origen y puerto de destino, mientras que un socket UDP se identifica simplemente por la IP y el puerto de origen.

Conviene usar TCP cuando la aplicación necesita que todos los datos lleguen correctamente. Como por ejemplo correo electrónico, Web, transferencia de archivos, etc. aplicaciones que valoran la fiabilidad por encima de tiempo o sobrecarga.

En cambio, conviene usar UDP cuando la aplicación tolera cierta pérdida, pero necesita evitar sobrecarga. Como por ejemplo voz o multimedia en tiempo real.

5.5 Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.

El protocolo desarrollado no cuenta con un algoritmo de control de congestión ni de control de flujo. Sí se provee un mecanismo de ventana deslizante, pero que es propio de Selective Repeat y no es una estrategia de control de flujo ni de congestión.

Es más, el protocolo retransmite indefinidamente los segmentos que caigan en timeout, y no sondea la red para revisar su estado. Funciona independientemente del estado de la red o del host peer.

5.6 ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

Stop and Wait tiene una ventana de 1 paquetes, por lo tanto se queda "congelado" esperando el ACK de ese. Hasta que no lo reciba, no puede enviar el siguiente.

Mientras que el Selective Repeat tiene una ventana de N paquetes, lo que le permite enviar varios paquetes antes de recibir el ACK. En caso de que se pierda uno, se retransmite ese en específico.

Generalmente Selective Repeat es superior en velocidad. Un caso excepcional sería el de transmisión de un archivo que entre en un único segmento de protocolo, en cuyo caso sería indistinto transmitirlo por Selective Repeat o Stop Wait.

6 Dificultades Encontradas

6.1 Definición de la Arquitectura de Concurrencia

Durante las etapas iniciales del desarrollo, uno de los mayores desafíos fue definir el modelo de concurrencia más adecuado para el servidor. Sabíamos que necesitábamos soportar múltiples transferencias en simultáneo, pero dudamos qué diseño arquitectónico adoptar.

Inicialmente, debatimos entre tres enfoques principales:

- El uso de **Tasks** (tareas asíncronas / *async-await*), que ofrecían un manejo eficiente de la I/O pero delegaban gran parte del control al *runtime*.
- La implementación del **Modelo de Actores**, el cual resultaba atractivo para evitar el estado compartido, pero añadía una curva de aprendizaje y una complejidad estructural quizás excesiva para el alcance del trabajo.
- El manejo directo de **Hilos** (*threads*) nativos del sistema operativo.

El problema principal era encontrar un equilibrio entre la simplicidad de implementación, el control de la concurrencia y la prevención de condiciones de carrera al momento de escribir en disco. Finalmente, decidimos descartar las *Tasks* y los Actores para avanzar con una arquitectura multi-hilo tradicional basada en el patrón *Acceptor-Handler*, apoyándonos en *locks* dentro del `FileManager` para garantizar la integridad de los archivos de forma segura y determinista.

6.2 Instalación de Mininet

A la hora de instalar Mininet, se siguieron inicialmente las instrucciones provistas en la página oficial. Sin embargo, durante el proceso nos encontramos con múltiples problemas de dependencias.

Estos conflictos estuvieron relacionados principalmente con la incompatibilidad de versiones entre los paquetes actuales del sistema operativo (como las versiones recientes de Python y ciertas librerías de red) y los requisitos legados del entorno de Mininet. Fue necesario realizar una investigación y resolución manual de las dependencias, ajustando la configuración del entorno y forzando la instalación de los paquetes correctos.

7 Conclusión

Se implementó un protocolo de transferencia de archivos sobre UDP. A través de este proceso, se logró transformar un medio de transporte no confiable en un canal seguro y ordenado, mediante el desarrollo de una arquitectura de capas, una máquina de estados finitos y mecanismos de transferencia confiable.

Las principales conclusiones derivadas del desarrollo son:

- **Arquitectura de Software:** El uso del Patrón *State* y el Patrón *Factory* resultó determinante para gestionar la complejidad lógica. Esto permitió implementar dos protocolos distintos (*Stop-and-Wait* y *Selective Repeat*) bajo una misma interfaz, asegurando un diseño modular, escalable y fácil de mantener.
- **Concurrencia y Robustez:** La implementación de *Acceptor-Handler* y el uso de colas de eventos (*Queue*) demostraron ser eficaces para manejar múltiples sesiones concurrentes sin sacrificar la estabilidad del servidor. El uso de *locks* en la gestión de archivos y la pre-asignación de recursos en disco garantizaron la integridad de la persistencia frente a accesos simultáneos.
- **Resiliencia ante fallos:** La integración de mecanismos de control como los temporizadores (*Timer*), el cálculo de *checksums* y la ventana deslizante permitió cumplir con el requisito de entrega garantizada incluso ante una tasa de pérdida del 10%. Esto valida que el protocolo es capaz de recuperar información y gestionar cierres ordenados bajo condiciones de red adversas.